

Adaptive Camera Cleaning System

What it is, what I built, and how every component works. UIUC ECE 445 senior design, Jan to May 2025, team of three. Repo: github.com/dbhan08/Weather-Resistant-Camera

1. What the system does

A weather-resistant enclosure keeps a camera lens usable in rain and frost. An STM32 microcontroller captures frames from a small camera, runs an int8 convolutional neural network under TensorFlow Lite Micro to decide whether the lens is wet, and drives a servo wiper when it is. The same MCU reads its temperature sensor and switches a heating pad through a MOSFET when the temperature drops below a threshold. Everything runs on the board. Once powered, no computer is needed.

Final numbers from the demo and test set:

Metric	Value
Rain / ice detection accuracy on the 100-image test set	89 %
Wipe trigger	5 % of frame tiles classed raindrop
Heater trigger	temperature below threshold (20.5 C in the demo, using an ice cooler)
Outdoor field test	2 hours continuous
Model on the MCU	about 20 k parameters, int8, six TFLite Micro ops

Seven subsystems: heater, temperature sensor, microcontroller, wiper, power, camera, and vision. I owned the camera and vision subsystems end to end and the integration between them and the MCU.

2. What I was responsible for

- **Camera subsystem.** Selected the camera modules (DFRobot FIT0701 for bench work, OV7670 on the board), wrote the OpenCV capture path for development and the DCMI + DMA capture driver on the STM32.
- **Vision subsystem.** Built the raindrop detector: an early OpenCV heuristic for bring-up, then the CNN that shipped: a ported pretrained raindrop classifier, fine-tuned, distilled into a microcontroller sized student, quantized to int8, and run under TensorFlow Lite Micro on the STM32.
- **Integration.** Wrote the firmware control loop that ties temperature, camera, CNN, servo and heater together, the serial protocol used during development, and the status logging used to score the test set.
- **Documents.** Introduction, Problem Statement and Proposed Solution in the proposal and design document; Requirements and Verification for every subsystem and the high-level requirements in the final paper, including all verification data.
- **Shared.** PCB iterations in KiCad, machine-shop enclosure work, soldering and bring-up.

3. Hardware

Block	Part / peripheral	Role
MCU	STM32F4; custom PCB, Nucleo dev board for the demo	Control loop, PWM, ADC, UART, DCMI, CNN inference

Camera	OV7670 on DCMI + DMA2, SCCB over I2C1, XCLK from a timer	160x120 RGB565 frames into SRAM
Wiper	Hobby servo on TIM2 CH1 PWM	60 to 115 degree sweep
Heater	Heating pad, N-MOSFET on PA5, level shifter, indicator LED	On below the temperature threshold
Sensor	STM32 internal temperature sensor on ADC1	Factory calibrated at 30 C and 110 C
Power	5 V and 3.3 V regulators	Servo and heater at 5 V, MCU at 3.3 V
Debug	USART2 at 115200 baud	Status log; START / STOPP during development

The PCB went through five KiCad revisions. Changes along the way: added a USB connector and crystal oscillators, swapped the 5 V regulator, moved to the course's STM32 reference layout, replaced the USB-powered heater with a power-and-ground pad switched by a MOSFET, added a 3.3 V to 5 V level shifter for the MOSFET gate, and added LEDs for the heater state. The first board had missing pull-ups found after ordering; the second board's MCU was destroyed while hand-soldering its fine-pitch package, so the demo ran the same firmware on the Nucleo.

4. Firmware, component by component

4.1 Servo wiper (TIM2 PWM)

The servo wants a pulse of roughly 1 to 2 ms every 20 ms. TIM2 channel 1 is set up in CubeMX as PWM mode 1 with a period that makes 0 to 180 degrees map onto compare values 210 to 1050. One helper does the map:

```
void Set_Servo_Angle(TIM_HandleTypeDef *htim, uint32_t ch, uint8_t angle) {
    uint32_t pulse = 210 + (angle * (1050 - 210) / 180);
    __HAL_TIM_SET_COMPARE(htim, ch, pulse);
}
static void Wipe(void) {
    for (uint8_t a = 60; a <= 115; a += 3) { Set_Servo_Angle(&htim2, TIM_CHANNEL_1, a); HAL_Delay(100); }
    for (uint8_t a = 115; a > 60; a -= 3) { Set_Servo_Angle(&htim2, TIM_CHANNEL_1, a); HAL_Delay(100); }
}
```

A sweep is 60 to 115 degrees and back in 3-degree steps at 100 ms per step, about 3.7 s. The step delay keeps the wiper arm from slamming the lens housing.

4.2 Temperature sensor (ADC1) and heater (PA5 MOSFET)

The STM32 has a temperature sensor on an internal ADC channel. Two calibration values burned into system memory at the factory give the raw ADC reading at 30 C and 110 C, so the conversion is a straight line between them. A 9 C offset was found by comparing against a reference thermometer.

```
#define TS_CAL1 (*((uint16_t*)0xFF7A2C)) // raw ADC at 30 C
#define TS_CAL2 (*((uint16_t*)0xFF7A2E)) // raw ADC at 110 C
HAL_ADC_Start(&hadc1); HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY);
raw = HAL_ADC_GetValue(&hadc1); HAL_ADC_Stop(&hadc1);
t = ((float)raw - TS_CAL1) * (110.0f - 30.0f) / ((float)TS_CAL2 - TS_CAL1) + 30.0f - 9.0f;
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, t < HEATER_THRESH_C ? GPIO_PIN_RESET : GPIO_PIN_SET);
```

Pulling PA5 low turns the heater on: the MOSFET is wired so that the MCU signal, through the level shifter, removes the voltage that keeps the pad off. The heater uses only the thermistor. Vision never drives it, because many things look like ice on a lens but temperature is unambiguous. The demo used a cooler of ice, which cannot reach 0 C, so the threshold was set at 20.5 C to show the behaviour.

4.3 Camera capture (DCMI + DMA)

The OV7670 is configured over SCCB, an I2C-like bus at address 0x42: reset, QVGA RGB mode, RGB565 output, the down-scaler enabled with a /4 divider in both axes to give 160x120, and automatic gain, white balance and exposure. Its pixel clock, HREF and VSYNC feed the DCMI peripheral, which DMA's one frame into SRAM in snapshot mode. The capture blocks on the frame-complete callback with a 500 ms timeout, then expands RGB565 to RGB888, replicating the high bits into the low bits so full white is 255 and not 248.

```
static const uint8_t k_ov7670_init[][2] = {{0x12, 0x80}, {0x12, 0x14}, {0x40, 0xD0}, {0x0C, 0x04},
    {0x3E, 0x1A}, {0x72, 0x22}, {0x73, 0xF2}, {0x13, 0xE7}, ...};
HAL_DCMI_Start_DMA(&hdcmi, DCMI_MODE_SNAPSHOT, (uint32_t)s_frame565, (CAM_W * CAM_H * 2) / 4);
while (!s_frame_done) if (HAL_GetTick() - t0 > 500) return 0;
rgb888[3*i] = (r5 << 3) | (r5 >> 2); // g6 << 2 | g6 >> 4; b5 << 3 | b5 >> 2
```

160x120 was chosen because it fits the F4's 192 KB SRAM next to the model's tensor arena: 38 KB for the RGB565 DMA buffer and 58 KB for RGB888. It gives a 3 by 2 grid of 48 px tiles.

5. Vision: the CNN

The detector is a **tile classifier**. The frame is cut into a fixed grid of 48 px tiles. Each tile is labelled raindrop or clean by the network, and the fraction of raindrop tiles is the coverage number that triggers the wiper. This is the coverage-metric idea from automotive lens-soiling work (Valeo SoilingNet and TiledSoilingNet). It is cheap enough for a microcontroller, and the grid also says where on the lens the water is.

5.1 Starting point: a published pretrained model

No public model exists for water on a lens, so I started from the closest one: the AlexNet-30² raindrop classifier from Guo, Akcay, Adey and Breckon (ICIP 2018), 0.95 accuracy on 30x30 patches from a windshield camera, MIT licensed. Its weights are a TensorFlow 1 / TFLearn checkpoint that no current toolchain can run or convert. TensorFlow 2 can still read the raw variables, so *port_breckon.py* rebuilds the same graph in Keras layer for layer and copies every tensor by name, asserting each shape. The architecture is five convolutions (96 at 11x11 stride 4, 256 at 5x5, 384, 384, 256 at 3x3) with 3x3 max-pools and local response normalization, then two 4096-wide tanh dense layers and a 2-way softmax.

```
_NAME_MAP = {"conv1": "Conv2D", "conv2": "Conv2D_1", ..., "fc3": "FullyConnected_2"}
reader = tf.train.load_checkpoint(tfl_path)
w, b = reader.get_tensor(f"{scope}/w"), reader.get_tensor(f"{scope}/b")
assert w.shape == layer.get_weights()[0].shape
layer.set_weights([w, b])
```

5.2 Data and fine-tuning for our lens

Breckon's camera looked through a windshield. Ours is millimetres from the drops, so drops are larger and blurrier. I built a patch set from RaindropsOnWindshield (8,190 frames with pixel masks, CC BY 4.0) with *data_windshield.py*: a 30x30 patch is positive if the mask covers at least 30 % of it, and negative only if the patch and a one-patch margin around it are mask-free, so borderline drops never leak into the negatives. Splits are by video sequence so no clip appears in both train and test. Frames from our own camera were added the same way.

finetune.py does the standard two-stage transfer: dense head only at learning rate 1e-3 for a few epochs, then all layers at 1e-5 with early stopping on validation loss, with flips and 45-degree rotations as in the original paper.

5.3 Distilling to something that fits the MCU

After three pools a 30x30 input collapses to 1x1x256, followed by 256x4096 and 4096x4096 dense layers. That is about 18 million parameters, 75 MB in fp32 and 19 MB in int8, against 512 KB of flash. The model also uses local response normalization, which TensorFlow Lite Micro does not implement. So the fine-tuned network is the **teacher** on the PC and a small **student** is what runs on the chip.

The student (*distill_student.py*) takes a 48x48 tile through a stem convolution and five depthwise-separable blocks (MobileNet style), global average pooling, and two logits. About 20 k parameters, ReLU only, and only ops TFLM has: CONV_2D, DEPTHWISE_CONV_2D, MEAN, FULLY_CONNECTED. It is trained by knowledge distillation (Hinton et al. 2015): KL divergence to the teacher's probabilities softened at temperature 4, mixed with cross-entropy on the hard label. The soft labels carry what the teacher knows about glare and texture, which was exactly the failure mode of the earlier heuristic.

```
x = _cbr(inp, 16, 3, 2)          # 48 -> 24
x = ds_block(x, 32, 2); x = ds_block(x, 32, 1)  # 24 -> 12
x = ds_block(x, 64, 2); x = ds_block(x, 64, 1)  # 12 -> 6
x = ds_block(x, 128, 2)          # 6 -> 3
x = GlobalAveragePooling2D()(x); out = Dense(2)(x)
loss = alpha * KL(teacher_T, student_T) * T**2 + (1 - alpha) * CE(y, logits)  # T=4, alpha=0.7
```

5.4 Quantization and export

to_tflite.py converts the student to full-integer int8: int8 weights, activations, input and output. The converter runs a representative set of a few hundred validation tiles through the fp32 graph to record every tensor's range. Validation tiles, not training tiles, so the ranges are not tuned on the images the weights were fitted to. The script writes the model as a C array (*raindrop_student_int8.cc*) and a report with byte size, op list, input scale and zero point, and int8-versus-fp32 agreement.

```
conv.optimizations = [tf.lite.Optimize.DEFAULT]
conv.representative_dataset = lambda: ([x[None]],) for x in rep)
conv.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
conv.inference_input_type = conv.inference_output_type = tf.int8
```

5.5 Inference on the STM32 (*raindrop_inference.cc*)

At boot, *RaindropInit* loads the model from the flash array, registers just the six ops in a *MicroMutableOpResolver*, allocates tensors in a 48 KB arena, and checks the input is int8 48x48x3 with a 2-way output. Per frame, *RaindropCoverage* walks the 3x2 tile grid. Each tile is copied out of the frame row by row and quantized with the scale and zero point the converter stored in the model. The student was trained on pixel/255, so:

```
q = round((pixel / 255) / input_scale) + input_zero_point, clamped to [-128, 127]
...
g_interp->Invoke();
raindrop = out[1] > out[0]; // argmax on int8 logits is order-preserving
return hits / (rows * cols); // coverage in [0, 1]
```

The Cortex-M DWT cycle counter times each pass and the value is printed on the status line, which is how per-frame latency was recorded.

6. Integration: the control loop (main_raindrop.c)

One loop, once per second, ties every component together:

- 1 Read the temperature through ADC1. Below the threshold, pull PA5 low to turn the heater on; the LED shows the state.
- 2 Capture a 160x120 frame over DCMI.
- 3 Run the CNN over the 3x2 grid and get the coverage fraction.
- 4 If coverage is at least 5 %, the temperature is above 0 C, and 4 s have passed since the last wipe, sweep the servo. The 0 C lockout stops the wiper from fighting ice; the cooldown stops re-triggering during a sweep.
- 5 Print one status line on USART2: temperature, heater state, coverage, the six-character tile grid, inference microseconds, and whether a wipe ran.

```
for (;;) {
    float temp_c = ReadTemperatureC();
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, temp_c < HEATER_THRESH_C ? GPIO_PIN_RESET : GPIO_PIN_SET);
    float cover = -1.0f;
    if (cam_ok && nn_ok && CameraCaptureRGB888(s_frame))
        cover = RaindropCoverage(s_frame, CAM_W, CAM_H, s_grid);
    if (cover >= WIPE_THRESHOLD && temp_c > 0.0f && HAL_GetTick() - last_wipe > WIPE_COOLDOWN_MS) {
        Wipe(); last_wipe = HAL_GetTick();
    }
    printf("T=%.1fC heat=%d cover=%.2f grid=%s nn_us=%lu\r\n", ...);
    HAL_Delay(LOOP_PERIOD_MS - spent);
}
```

Memory budget on the F4: 38 KB RGB565 frame, 58 KB RGB888 frame, 48 KB tensor arena, about 25 KB of model in flash. Build: enable DCMI, DMA2, I2C1 and a camera clock output in the .ioc; import a tflite-micro tree generated with *create_tflm_tree.py*, compiled as C++17 with TF_LITE_STATIC_MEMORY and CMSIS_NN; add the three firmware files and the model array; link with nano.specs and -u _printf_float.

6.1 The development path before the on-chip loop

During bring-up the MCU was driven from a laptop. *final_code.py* grabbed frames with OpenCV and sent a 5-byte command over USART2; the firmware blocked up to 1 s on a 5-byte receive and ran the sweep on START. The very first detector on that path was an HSV heuristic: keep pixels with saturation below 40 and value above 100, morphological close, enclose the largest contours in circles, count coverage. It false-fired on glare and texture, and the TA helped find why: the lens is so close to the wet surface that small texture is magnified. That is what pushed the design to a learned tile classifier trained on real drops, and then onto the MCU so the laptop could go away.

7. Test and verification

- **Detection accuracy.** 100-image test set of rain and ice on the lens under varied lighting, scored by comparing the printed tile grid against the lens: **89 %**.
- **Subsystems.** Regulators at 5 V and 3.3 V; MOSFET output with and without gate drive; servo on a servo tester (found a dead motor); temperature against a reference thermometer, giving the 9 C offset; PWM verified on a scope.
- **Model pipeline.** Every stage writes a JSON report: port (per-variable match, sample-patch predictions), fine-tune and distillation (test accuracy), TFLite (int8 vs fp32 agreement, byte size, op list). The PyTorch fallback pipeline in *lens_soiling/* has nine pytest unit and integration tests covering train, export and infer.

- **End to end.** Final demo with all subsystems interacting: freezing simulated with an ice cooler and the 20.5 C threshold, wiper triggered by detected water, LED confirming heater state. Two-hour outdoor field test of the enclosure.

8. Problems and how they were handled

Problem	Cause	Fix
Heuristic false positives on glare and texture	Lens millimetres from the drop surface, so texture is magnified	Learned tile classifier trained on real drops; distillation carries glare cases from the teacher
Pretrained model cannot run on the STM32	18 M params; LRN op not in TFLM	Knowledge distillation into a 20 k-param student using only TFLM ops
Pretrained weights unusable	TF1 / TFLearn checkpoint	Read raw variables with <code>tf.train.load_checkpoint</code> , rebuild in Keras, copy by name
Frame plus model did not fit SRAM at QVGA	192 KB SRAM on the F4	QQVGA 160x120, 48 KB arena, six-op resolver
Second PCB MCU destroyed	Fine-pitch package, hand soldering	Same firmware demoed on the Nucleo dev board
Servo not moving	Dead motor	Confirmed on a servo tester, replaced
No freezing conditions indoors	Bench test	Ice cooler, heater threshold raised to 20.5 C

9. Interview talking points

- **Tradeoff.** Tile classification with a coverage number instead of segmentation: far cheaper on an MCU and it maps straight onto the wipe rule. You lose drop shape, which you do not need.
- **Tradeoff.** Distill a published model rather than train small from scratch: reuse its knowledge of glare and skip collecting a large dataset, at the cost of a two-model pipeline.
- **Design choice.** Heater on the thermistor, wiper on vision, 0 C lockout so they never conflict.
- **Broke.** The near-lens geometry that made the heuristic false-fire, found by looking at the mechanical setup with the TA rather than at the code.
- **Next time.** Mount the camera farther from the wet surface. Add a frost class to the student. Move to an STM32H7 for a full QVGA frame and a wider student.

References: Guo, Akcay, Adey, Breckon, ICIP 2018 (github.com/tobybreckon/raindrop-detection-cnn). Soboleva & Shipitko, RaindropsOnWindshield, 2021 (Zenodo 4680442). Uricar et al., SoilingNet, ITSC 2019. Hinton, Vinyals, Dean, Distilling the Knowledge in a Neural Network, 2015. Howard et al., MobileNets, 2017.